

1 Key Concepts

1.1 Proof Techniques

Direct proof. This is used for proving *implications*, i.e., statements of the form $P \implies Q$ (i.e., “if P , then Q ”). We assume that P is true, then write a sequence of logical consequences *directly* showing that Q must also be true.

Proof by contrapositive. This is also used for proving implications $P \implies Q$. But in this case we directly prove the logically equivalent *contrapositive* of the statement, which is $\neg Q \implies \neg P$. That is, we assume that $\neg Q$ is true, then derive $\neg P$ as a logical consequence.

Proof by contradiction. Suppose we want to prove a statement P . We can do this by first assuming that P is false, then deriving some logical contradiction. Since the assumption that P is false led to a contradiction, it must be that P is true.

1.2 Induction

Induction is used to prove a statement about a countably infinite set, such as the positive integers. To prove a statement $P(n)$ about the positive integers, we show two things:

- (1) $P(1)$ is true.
- (2) If k is any positive integer and $P(k)$ is true, then $P(k+1)$ is true as well.

If we can show these two things, then we can conclude from (1) that $P(1)$ is true. Then, using (1) and (2) together, we conclude that $P(2)$ is true, and then so is $P(3)$, and so on.

Strong Induction Strong induction differs from typical induction in that, during the inductive step, typical induction proves a claim of the form $P(k) \rightarrow P(k+1)$ whereas strong induction proves a claim of the form $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$.

1.3 Asymptotic Notation

In the following, let $f(n)$ and $g(n)$ be positive functions. We define the following asymptotic notations.

Definition 1. We say that $f(n) = O(g(n))$ if there exist positive constants $c, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Definition 2. We say that $f(n) = \Omega(g(n))$ if there exist positive constants $c, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

(Notice that the only difference between [Definitions 1](#) and [2](#) is the $\leq$ versus the $\geq$ comparison.)

Definition 3. We say that $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Lemma 4. When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, $f(n) = O(g(n))$ if and only if the limit is finite.

Note: it is not *necessary* that the limit exists: even if it does not, it may still be the case that $f(n) = O(g(n))$.

Lemma 5. When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, $f(n) = \Omega(g(n))$ if and only if the limit is greater than zero (including infinite).

Again, it is not *necessary* that the limit exists: even if it does not, it may still be the case that $f(n) = \Omega(g(n))$.

1.4 The Potential Method

The potential method is a useful technique to reason about the amount of time an algorithm takes before halting. By defining an appropriate potential function for an algorithm, and establishing how fast it decreases and a lower bound for it, we can get an upper bound on the number of “units of work” the algorithm performs (as a function of its input).

Formally, we define a function $s: A \rightarrow \mathbb{R}$ that maps the “state” of the algorithm (i.e., its internal variables) to a (finite) real number. The function s is a valid potential function if:

1. s decreases by some *fixed positive constant* $C > 0$ with every “unit of work” done by the algorithm—for a suitable notion of “work unit”, e.g., a loop iteration;
2. s has some *lower bound* B , i.e., the algorithm is guaranteed to terminate once $s \leq B$.

The first condition ensures that $s \leq B$ will eventually occur within a finite number of work units: specifically, $(I - B)/C$ units, where I is the initial value of the potential function (for the original input to the algorithm). The second condition then ensures that the algorithm will *terminate* within that number of work units.

2 Practice Problems

1. Comparing asymptotic running times.

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

Solution:

Some.

Suppose that an algorithm A takes exactly $376n \log n = O(n \log n)$ steps on any input of size n . Suppose that another algorithm B takes exactly $n^2 = O(n^2)$ steps on any size- n input. Then for small n (say, $n = 2$), algorithm B takes fewer steps than algorithm A does. Specifically for $n = 2$, A would take $376 \cdot 2 \cdot 1 = 752$ steps, whereas B would take $2^2 = 4$ steps. In this case, the $O(n \log n)$ -time algorithm takes more time than the $O(n^2)$ -time algorithm.

Now suppose that an algorithm A takes exactly $n \log n = O(n \log n)$ steps on any size- n input, and that another algorithm B takes $376n^2 = O(n^2)$ steps. Then for *all* $n \geq 1$, algorithm A takes fewer steps than algorithm B does, because $\log n \leq 376n$. For example, for $n = 2$, A would take $2 \cdot 1 = 2$ steps whereas B would take $376 \cdot 2^2 = 1504$ steps. In this case the $O(n^2)$ -time algorithm takes more time than the $O(n \log n)$ -time algorithm, on *every* input.

2. Asymptotics.

For the following pairs of functions $f(n)$ and $g(n)$, is $f(n) = O(g(n))$? Justify your answer by applying the definition of big-O or by applying a limit argument.

- $f(n) = \log_a n$, $g(n) = \log_b n$ for constants $a, b > 1$.

Solution: Yes. Setting $c = \frac{\ln b}{\ln a}$ and $n_0 = 1$, we have that

$$\begin{aligned} f(n) &= \log_a n \\ &= \frac{\ln n}{\ln a} \\ &= \frac{\ln n}{\ln a} \cdot \frac{\ln b}{\ln b} \\ &= \frac{\ln b}{\ln a} \cdot \frac{\ln n}{\ln b} \\ &= \frac{\ln b}{\ln a} \cdot g(n), \end{aligned}$$

as required by [Definition 1](#). (Note that $=$ implies $\leq$, which is all that's needed by the definition.)

- $f(n) = 4^n$, $g(n) = 2^n$

Solution: No. We will show that $f(n) \neq O(g(n))$ by giving a direct proof of the negation of [Definition 1](#), which is:

for *every* positive constants $c, n_0 > 0$, *there exists* some $n \geq n_0$ such that $f(n) > c \cdot g(n)$.

(Remember that when negating a statement that has “there exist(s)” and/or “for all” quantifiers, the former becomes the latter and vice versa.)

So, let $c, n_0 > 0$ be some arbitrary positive constants. Then consider some arbitrary $n > \max\{\log_2 c, n_0\}$, i.e., $n > \log_2 c$ and $n > n_0$. For such n , we have that

$$f(n) = 4^n = (2^n) \cdot (2^n) > (2^{\log_2 c}) \cdot (2^n) = c \cdot 2^n = c \cdot g(n),$$

as needed. (In fact, we have shown that $f(n) > c \cdot g(n)$ for *every* $n > \max\{\log_2 c, n_0\}$, which is more than what's required to prove the negation of [Definition 1](#), but it certainly suffices).

We can also show this using [Lemma 4](#):

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} &= \lim_{n \rightarrow \infty} \frac{2^n \cdot 2^n}{2^n} \\ &= \lim_{n \rightarrow \infty} 2^n \\ &= \infty. \end{aligned}$$

Thus, $f(n) \neq O(g(n))$.

3. Faulty proofs.

For each of the proposed “proofs” of the following (true!) statements, determine what is incorrect about the proof.

- (a) Claim: There is no even prime greater than 2. Proof: 3 is a prime that is greater than 2 and it is not even, so there must be no even prime greater than 2.

Solution:

What needs to be shown is that there *does not exist* any prime greater than 2 that is even, or in other words, *every* prime greater than 2 is odd. However, this proof gave just *one example* of an odd prime greater than 2. In general, we cannot prove a “for all” statement by showing that the statement holds for just one example.

- (b) Claim: If n^2 is even, then n is even. Proof: If n is even, then $n = 2k$ for some integer k . Therefore, $n^2 = 4k^2 = 2 \cdot 2k^2$ is even.

Solution: The proof attempts to prove a statement of the form $p \implies q$ by proving its converse $q \implies p$; this is not logically valid. While in some cases both $p \rightarrow q$ and $q \rightarrow p$ are true, in other cases only one of them is true. (For example, “if shape S is a square, then S is a rectangle” is true, but the converse “if S is a rectangle, then S is a square” is not true.) So in general, $q \rightarrow p$ being true does not imply that $p \rightarrow q$ is true.

Instead, this claim could be proven either directly, or via the contrapositive (“if n is odd, then n^2 is odd”).

- (c) Claim: The sum of the first i positive odd integers equals the i th square. Proof: $1 = 1^2$, and $1 + 3 = 4 = 2^2$, and $1 + 3 + 5 = 9 = 3^2$, and so on. The pattern will continue, so the claim is proven.

Solution: It is not logical to show that a claim holds in a few small instances, then conclude that it must hold in all instances. Noticing that there is a pattern and becoming convinced that the claim is true can be very helpful for developing intuition and starting to conceive of a proof, but they do not constitute a proof itself. One valid way to prove the given claim is through induction.

- (d) Claim: 37 is the largest prime smaller than 41. Proof: 37 is less than 41, and is prime since its only divisors are 1 and 37.

Solution: This is not a valid proof because the proof did not demonstrate that 37 is the *largest* prime smaller than 41; it showed only that 37 is prime and less than 41. To complete the proof, one should also show that there is no prime strictly between 37 and 41, i.e., that 38, 39 and 40 are all composite.

- (e) Claim: The sum of two even integers is even. Proof: Let n, m be any two even integers. Since they are both even, then let $n = 2k$ and $m = 2k$ for some integer k . $n + m = 2k + 2k = 4k = 2(2k)$, therefore the sum is also even.

Solution: This proof is incorrect because it implicitly assumes that n and m are the *same* even number (when it says “let $n = 2k$ and $m = 2k$ ”). This is not necessarily the case, since n, m are *arbitrary* even numbers. (A related red flag is the two uses of “let” for n, m : once we have let n, m be arbitrary even numbers, we cannot later re-assign them or assume anything additional about them.)

A correct proof goes like this: since n, m are even, we know that $n = 2\ell$ and $m = 2k$ for some integers ℓ, k . Then $n + m = 2\ell + 2k = 2(\ell + k)$ is even, as claimed.

4. Proof by contradiction.

Prove by contradiction that $\sqrt{7}$ is irrational.

Solution: Since this proof is by contradiction, the first step is to assume that the negation of the claim is true, and then proceed to derive a contradiction.

So, assume that $\sqrt{7}$ is rational.

By definition, this means that $\sqrt{7} = a/b$ for some integers a, b . Moreover, we can assume that a and b share no common factor greater than 1, because otherwise we can cancel out such a factor without affecting the value of a/b .

Then $7 = a^2/b^2$ and thus $7b^2 = a^2$. Since a^2 is divisible by 7, which is prime, it must also be true that a is divisible by 7. Therefore, $a = 7k$ for some integer k .

Substituting $a = 7k$, we get that $7b^2 = (7k)^2 = 49k^2$. Dividing by 7, $b^2 = 7k^2$. This then implies that b^2 is divisible by 7, and thus that b is divisible by 7 by the same argument as above.

We have shown that a and b are both divisible by 7, but we also showed that they have no common factor greater than one. This is a contradiction! So, our original assumption—that $\sqrt{7}$ is rational—must be false, hence $\sqrt{7}$ is irrational, as claimed.

5. Euclid’s algorithm.

Which pair of numbers takes the greatest number of iterations to find the GCD when using Euclid’s algorithm?

- ☐ (800, 300)
- ☐ (100, 55)
- ☒ (29, 18)
- ☐ (1000, 999)
- ☐ (19, 11)

Solution: $\gcd(29, 18)$ takes the most iterations, as we can see by showing the sequence for each pair:

$$\gcd(800, 300) = \gcd(300, 200) = \gcd(200, 100) = \gcd(100, 0) = 100$$

$$\gcd(100, 55) = \gcd(55, 45) = \gcd(45, 10) = \gcd(10, 5) = \gcd(5, 0) = 5$$

$$\gcd(29, 18) = \gcd(18, 11) = \gcd(11, 7) = \gcd(7, 4) = \gcd(4, 3) = \gcd(3, 1) = 1$$

$$\gcd(1000, 999) = \gcd(999, 1) = 1$$

$$\gcd(19, 11) = \gcd(11, 8) = \gcd(8, 3) = \gcd(3, 2) = \gcd(2, 1) = 1$$

6. Induction

Let $a_1, a_1r, a_1r^2, \dots, a_1r^n$ be a finite geometric sequence with common ratio r and initial term a_1 (notice the 1-indexing). Prove using induction that the sum of the i terms is equal to $S_i = \frac{a_1(1-r^i)}{1-r}$.

Solution:

Let the proposition $P(k)$ be true if and only if $S_k = \frac{a_1(1-r^k)}{1-r}$.

Base Case: $P(1) : \frac{a_1(1-r^1)}{1-r} = a_1 \cdot \frac{(1-r)}{1-r} = a_1$.

Inductive Step: Suppose that $P(k)$ is true for some positive integer k . We now want to show that $P(k+1)$ is also true. Note that the $k+1^{th}$ term equals a_1r^k rather than a_1r^{k+1} .

$$S_{k+1} = S_k + a_1 \cdot r^k \tag{1}$$

$$= \frac{a_1(1-r^k)}{1-r} + a_1 \cdot r^k \tag{2}$$

$$= \frac{a_1 - a_1 \cdot r^k + a_1 \cdot r^k - a_1 \cdot r^{k+1}}{1-r} \tag{3}$$

$$= \frac{a_1 - a_1 \cdot r^{k+1}}{1-r} \tag{4}$$

$$= \frac{a_1(1-r^{k+1})}{1-r} \tag{5}$$

Since $P(k) \rightarrow P(k+1)$ and since $P(1)$ was true, then by induction the original claim is also true.

7. Strong Induction

Suppose there is a pile of n stones that must be split into n piles of 1 stone each. Each split is characterized by taking a pile with > 1 stone and splitting the pile into 2 smaller piles each having at least 1 stone. Prove that no matter how the splits are performed over an initial pile

of size n (i.e. any ordering, equal/unequal splits, etc.), the sum of the pairwise products of the splits will equal $\frac{n(n-1)}{2}$.

As an example of the process, suppose there are 4 stones to start. One possible way to perform the splits is to have 3 splits as follows: $(2 - 2), (1 - 1) - 2, 1 - 1 - (1 - 1)$. This notation is intended to indicate first splitting into 2 piles of 2, then splitting 1 of the 2 piles of 2 into 2 piles of 1, and finally splitting the other pile of 2 into 2 piles of 1. Computing the sum of the pairwise products of the splits, we have $2 \cdot 2 = 4$ from the first split since the first split resulted in 2 piles of 2, then $1 \cdot 1 = 1$ from the second split since the second split resulted in 2 piles of size 1, and finally $1 \cdot 1 = 1$ for the third split. Summed together, we have $4 + 1 + 1 = 6 = \frac{4(4-1)}{2}$. The same process can be done by instead having the first split be $(3 - 1)$ and so on and the pairwise sum of splits will still be 6 (you can check!).

Solution: The claim can be proven using strong induction. As a minor note, strong induction differs from typical induction in that typical induction proves a claim during the inductive step of the form $P(k) \rightarrow P(k+1)$ whereas strong induction proves a claim of the form $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$.

Let the proposition $P(k)$ be true if and only if the sum of the pairwise product of splits of a pile of size k will equal $\frac{k(k-1)}{2}$ regardless of the way the splits are performed.

Base Case: $P(1)$: When starting out with a pile of size 1, there are no splits to be performed in order to reduce down to only piles of size 1 so the pairwise product of splits equals $0 = \frac{1(1-1)}{2}$.

Inductive Hypothesis: Suppose that $P(1) \wedge P(2) \wedge \dots \wedge P(k-1)$ is true.

Inductive Step: It is now the goal to prove that $P(k)$ is also true. The key insight is to perform a single arbitrary split on the pile of size k and then invoke the inductive hypothesis.

Given a pile of stones of size k , split the pile into one of size i and the other of size $k-i$ with $1 \leq i \leq k-1$. Also, let $SumOfSplits(n)$ be a function that maps pile size to the sum of splits of that pile size.

After performing the first split:

$$SumOfSplits(k) = SumOfSplits(i) + SumOfSplits(k-i) + i \cdot (k-i)$$

Then invoking the inductive hypothesis two times:

$$SumOfSplits(k) = \frac{i(i-1)}{2} + \frac{(k-i)(k-i-1)}{2} + i \cdot (k-i)$$

Performing algebra:

$$SumOfSplits(k) = \frac{i^2 - i + k^2 - 2ki + i^2 - k + i + 2ki - 2i^2}{2}$$

Canceling terms and simplifying:

$$\text{SumOfSplits}(k) = \frac{k^2 - k}{2} = \frac{k(k - 1)}{2}$$

Therefore, $P(k)$ is true. This completes the proof by strong induction to prove the original claim.

8. Potential functions.

Suppose *input* is a function that returns a user-specified positive integer. For each of the following problems, do the following tasks:

- Determine whether the following program halts for any sequence of (valid) inputs.
- If so, provide a proof using the potential method; if not, describe a sequence of inputs that causes the program to never halt.

(a)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

Solution: We claim that the value of $2x + y$ serves as a valid potential for this program.

Claim: The potential function, $2x + y$, has a lower bound. If the program doesn't exit the while loop on line 3, then $2x + y > 0$ (at that moment).

If the program doesn't exit the while loop, then we have $x > 0$ and $y > 0$. Thus $2x + y > 0$.

Claim: In each iteration of the while loop, $2x + y$ is strictly decreasing.

Suppose z is even. Then the value of the potential changes to $2(x - 1) + (y + 1) = 2x + y - 1 < 2x + y$.

Suppose z is odd. Then the value of the potential changes to $2x + (y - 1) < 2x + y$. In either case, the value of the potential function decreases by 1.

(b)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

Solution: The sequence $x \leftarrow 2$, $y \leftarrow 2$, and $z_i \leftarrow i$ (where z_i denotes the i -th input for z) causes the program to run forever. Notice that z_i at each i -th input alternates between even and odd. This causes the program to alternate between incrementing and decrementing x and y at each iteration of the while loop. Concretely, in our example (x, y) alternates between $(2, 2)$ and $(3, 1)$, never terminating.

(c)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  or  $y > 0$  do // This line differs from the program in (a)
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

Solution: The sequence $x \leftarrow 1$, $y \leftarrow 1$, and $z_i \leftarrow 1$ for all i causes the program to loop. The difference between this program and previous is that the loop only terminates if *both* x and y are non-positive. If $z_i \leftarrow 1$, only $y \leftarrow y - 1$ is executed, so x will remain positive, never terminating.